

Commit 849e037

Browse files

ActuallyNull committed last month

Model v1.3, spreadsheet updated, modified model architecture,decreased batch_size, improved data_processing and preperation algorithms

main 1 parent ac35ccb commit 849e037

Filter files...

- data_preperation.py
- data_processing.py
- model.py

3 files changed -61 -64 lines changed Search within code

```
data_preperation.py @@ -3,17 +3,12 @@
3 3 from sklearn.model_selection import train_test_split
4 4
5 5 def create_train_val_test_splits():
6 -
7 6 X = []
8 7 y = []
9 -
10 8 X_processed, y_processed = preprocess_dataset("training2017/training2017", "training2017/training2017/REFERENCE.csv",
11 - X, y)
12 9 X_cnn = X_processed.reshape((X_processed.shape[0], X_processed.shape[1], 1))
13 10 le = LabelEncoder()
14 11 y_cnn = le.fit_transform(y_processed)
15 -
16 - X_train, X_temp, y_train, y_temp = train_test_split(X_cnn, y_cnn, test_size=0.3, random_state=42)
17 - X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
18 -
12 + X_train, X_temp, y_train, y_temp = train_test_split(X_cnn, y_cnn, test_size=0.3, random_state=42, stratify=y_cnn)
13 + X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp)
19 14 return X_train, y_train, X_val, y_val, X_test, y_test
```

```
data_processing.py @@ -5,33 +5,26 @@
5 5 from sklearn.preprocessing import MinMaxScaler
6 6 from scipy.signal import butter, filtfilt, resample
7 7 from imblearn.over_sampling import SMOTE
8 - from collections import Counter
9 - from ecg_augmentor import ECG_Augmentor
10 8 + from ecg_augmentor import ECG_Augmentor
11 9
12 -
13 - dataset_folders = ["mit-bih", "training2017/training2017"]
14 - records = []
15 - labels = []
16 10 physionet_label_map = {'N': "Normal", 'A': "AFib", 'O': "Other Arrythmia"}
17 - duration = 10 #seconds
18 -
19 - X = []
20 - y = []
21 11 + duration = 10 # seconds
22 12
23 13 def butter_bandpass_filter(signal, fs, low_pass=0.5, high_pass=50, order=5):
24 14 nyq = 0.5 * fs
25 15 low = low_pass / nyq
26 16 high = high_pass / nyq
27 - b,a = butter(order, [low, high], btype='band')
28 17 + b,a = butter(order, [low, high], btype='band')
29 18 return filtfilt(b, a, signal)
30 19
31 20 def smote_resampling(X, y):
32 - smote = SMOTE(random_state=42)
33 - X_resampled, y_resampled = smote.fit_resample(X, y)
34 - return X_resampled, y_resampled
35 -
36 - def signal_resampling(signal, target_length):
37 - resample(signal, target_length)
38 21 + # Only apply SMOTE if there are at least two classes
39 22 + if len(np.unique(y)) > 1:
40 23 + smote = SMOTE(random_state=42)
41 24 + X_resampled, y_resampled = smote.fit_resample(X, y)
42 25 + return X_resampled, y_resampled
43 26 + else:
44 27 + return X, y
45 28
46 29 def standardise_dataset(X):
47 30 X_scaled = np.array([(X - np.mean(x)) / np.std(x) for x in X])
48 31 @@ -43,57 +36,45 @@ def normalise_dataset(X):
49 36 return X_norm
50 37
51 38 def load_physionet_dataset(physionet_path, reference_file_path, X, y):
52 -
53 39 reference_df = pd.read_csv(reference_file_path, header=None, names=['record', 'label'])
54 40 + records = []
55 41 + labels = []
56 42
57 - # loading the data
58 43 + # Only keep records that are not noisy
59 44 for file in os.listdir(physionet_path):
60 45 if file.endswith(".hea"):
61 46 record_name = file[:-4]
62 47 + label_row = reference_df.loc[reference_df['record'] == record_name]
63 48 + if label_row.empty:
64 49 + continue # Skip if not found in reference
65 50 + label_code = label_row['label'].values[0]
66 51 + if label_code == "-":
67 52 + continue # Skip noisy signals
68 53 records.append(record_name)
69 54 + labels.append(physionet_label_map.get(label_code, "Other Arrythmia"))
70 55
71 - for name in records:
72 - nameLabel = reference_df.loc[reference_df['record'] == name, 'label'].values[0]
73 - if nameLabel == "N":
74 - labels.append("Normal")
75 - elif nameLabel == "A":
76 - labels.append("AFib")
77 - else:
78 - labels.append("Other Arrythmia")
79 -
80 - # entering it in dataset
81 - for paths in records:
82 - record = wfdb.rdrecord(physionet_path + "\\ " + paths)
83 56 + # Load and pad signals
84 57 + for record_name in records:
85 58 + record = wfdb.rdrecord(os.path.join(physionet_path, record_name))
86 59
87 60 fs = record.fs
88 61 samples = fs * duration
89 62 ecg_signal = record.p_signal[:, 0]
90 63 trunc_ecg = ecg_signal[:samples]
91 - # Pad if shorter than 'samples'
92 64 if len(trunc_ecg) < samples:
93 65 trunc_ecg = np.pad(trunc_ecg, (0, samples - len(trunc_ecg)), 'constant')
94 66 X.append(trunc_ecg)
95 67
96 - for label in labels:
97 - y.append(label)
98 -
99 67 + y.extend(labels)
100 68 X = np.array(X, dtype=np.float32)
101 69 y = np.array(y)
102 70 return X, y
103 71
104 - def preprocess_dataset(physionet_path, reference_file_path, X, y, fs=300): #X and y should be empty lists
105 -
106 72 + def preprocess_dataset(physionet_path, reference_file_path, X, y, fs=300):
107 73 augmentor = ECG_Augmentor(fs=fs)
108 74
109 - X, y = load_physionet_dataset(physionet_path, reference_file_path, X, y)
110 75 X_filtered = np.array([butter_bandpass_filter(x, fs=fs) for x in X])
111 76
112 - # Augment each signal (single-signal augmentation)
113 77 X_augmented = np.array([augmentor.augment(x) for x in X_filtered])
114 78
115 - #X_standardised = standardise_dataset(X_augmented)
116 79 X_processed = normalise_dataset(X_augmented) # since this should be a dcnn, no standardisation needed
117 80 + X_processed = normalise_dataset(X_augmented) # For DCNN, normalization is preferred
118 81 y_processed = y
119 82
120 - # Apply SMOTE to balance the dataset
121 83
122 84 X_balanced, y_balanced = smote_resampling(X_processed, y_processed)
123 85 return X_balanced, y_balanced
```

```
model.py @@ -1,5 +1,5 @@
1 1 import tensorflow as tf
2 - from keras import layers
3 2 + from keras import layers, models
4 3 from keras.callbacks import EarlyStopping, ReduceLROnPlateau
5 4 from data_preperation import create_train_val_test_splits
6 5 from sklearn.metrics import confusion_matrix, classification_report
7 6 @@ -14,14 +14,37 @@
8 14 ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, min_lr=1e-6, verbose=1)
9 15 ]
10 16
11 - model = tf.keras.Sequential([
12 - layers.Conv1D(60, 1),
13 - layers.Conv1D(50, 1),
14 - layers.Conv1D(40, 1),
15 - layers.GlobalAveragePooling1D(),
16 - layers.Dense(20, activation='relu'),
17 - layers.Dense(3, activation='softmax')
18 - ])
19 17 + def ecg_cnn(input_length, num_classes=3): # 0: afib, 1: normal, 2: other arrhythmia
20 18 + inputs = layers.Input(shape=(input_length, 1))
21 19 +
22 20 + x = layers.Conv1D(32, kernel_size=15, padding='same')(inputs)
23 21 + x = layers.BatchNormalization()(x)
24 22 + x = layers.ReLU()(x)
25 23 + x = layers.MaxPooling1D(pool_size=2)(x)
26 24 +
27 25 + x = layers.Conv1D(64, kernel_size=11, padding='same')(x)
28 26 + x = layers.BatchNormalization()(x)
29 27 + x = layers.ReLU()(x)
30 28 + x = layers.MaxPooling1D(pool_size=2)(x)
31 29 +
32 30 + x = layers.Conv1D(128, kernel_size=7, padding='same')(x)
33 31 + x = layers.BatchNormalization()(x)
34 32 + x = layers.ReLU()(x)
35 33 + x = layers.MaxPooling1D(pool_size=2)(x)
36 34 +
37 35 + x = layers.Conv1D(256, kernel_size=5, padding='same')(x)
38 36 + x = layers.BatchNormalization()(x)
39 37 + x = layers.ReLU()(x)
40 38 + x = layers.GlobalAveragePooling1D()(x)
41 39 +
42 40 + x = layers.Dense(64, activation='relu')(x)
43 41 + x = layers.Dropout(0.5)(x)
44 42 + outputs = layers.Dense(num_classes, activation='softmax')(x)
45 43 +
46 44 + model = models.Model(inputs, outputs)
47 45 + return model
48 46 +
49 47 + model = ecg_cnn(input_length=3000) # 300 hz for 10 sec = 3000 samples
50 48
51 49 model.compile(optimizer='adam',
52 50 loss='sparse_categorical_crossentropy',
53 51 @@ -30,12 +53,10 @@
54 53 model.fit(X_train,
55 54 y_train,
56 55 epochs=100,
57 56 + batch_size=64,
58 57 callbacks=callback,
59 58 )
60 59
61 - print(model.summary)
62 -
63 60 test_loss, test_acc = model.evaluate(X_test, y_test)
64 61 print(f"Test accuracy: {test_acc:.4f}")
65 62
```

Comments 0

Please sign in to comment.